Angular Hands-On Exercise Solutions

HANDS-ON 1 — Environment Setup, Project Structure & First Component

Task 1: Scaffold and Explore the Angular Project

Setup commands

```
npm install -g @angular/cli
ng new student-course-portal --routing --style=css
cd student-course-portal
ng serve
```

notes.txt

```
angular.json        - workspace config, build/serve/test settings for all projects
tsconfig.json       - base TypeScript compiler options shared across the workspace
tsconfig.app.json   - TypeScript compiler options specific to the app build
package.json        - project dependencies, scripts (ng, start, build, test)
src/main.ts         - entry point, bootstraps the root AppComponent
src/app/app.config.ts - standalone app configuration (providers, router)
src/app/app.component.ts - root component of the application
src/index.html      - the single HTML page hosting the Angular app
```

ng build output

```
dist/student-course-portal/browser/main.js   - compiled application code
dist/student-course-portal/browser/index.html
dist/student-course-portal/browser/styles.css
```

angular.json budgets

```
"budgets": [
  {
    "type": "initial",
    "maximumWarning": "500kB",
    "maximumError": "1MB"
  },
  {
    "type": "anyComponentStyle",
    "maximumWarning": "4kB",
    "maximumError": "8kB"
  }
]
```

maximumWarning - build shows a warning once bundle size crosses this limit. maximumError - build fails once bundle size crosses this limit.

Task 2: Create and Organise Components

```
ng generate component components/header
ng generate component pages/home
ng generate component pages/course-list
ng generate component pages/student-profile
```

header.component.html

```
<nav>
  <span class="brand">Student Course Portal</span>
  <a routerLink="/">Home</a>
  <a routerLink="/courses">Courses</a>
  <a routerLink="/profile">Profile</a>
</nav>
```

home.component.ts

```
export class HomeComponent {
  coursesAvailable = 12;
  enrolled = 3;
  gpa = 3.8;
}
```

home.component.html

```
<h1>Welcome to Student Course Portal</h1>
<p>Track your courses, grades and enrollments in one place.</p>
<div class="stats-row">
  <span>Courses Available: {{ coursesAvailable }}</span>
  <span>Enrolled: {{ enrolled }}</span>
  <span>GPA: {{ gpa }}</span>
</div>
```

app.component.html

```
<app-header></app-header>
<router-outlet></router-outlet>
```

HANDS-ON 2 — Data Binding, Lifecycle Hooks & Component Communication

Task 1: All Four Binding Types

home.component.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
```

```
  selector: 'app-home',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './home.component.html'
})
export class HomeComponent {
  portalName = 'Student Course Portal';
  isPortalActive = true;
  message = '';
  searchTerm = '';

  onEnrollClick() {
    this.message = 'Enrollment opened!';
  }
}
```

home.component.html

```
<h1>{{ portalName }}</h1>

<button [disabled]="!isPortalActive" (click)="onEnrollClick()">Enroll Now</button>
<p>{{ message }}</p>

<input [(ngModel)]="searchTerm" placeholder="Search courses">
<p>Searching for: {{ searchTerm }}</p>

<!--
[property] is one-way: value flows from component to DOM only.
[(ngModel)] is two-way: value flows from component to DOM and DOM back to component.
-->
```

Task 2: Lifecycle Hooks

home.component.ts

```
import { Component, OnInit, OnDestroy } from '@angular/core';

export class HomeComponent implements OnInit, OnDestroy {
  ngOnInit() {
    console.log('HomeComponent initialised — courses loaded');
  }

  ngOnDestroy() {
    console.log('HomeComponent destroyed');
  }
}
```

course-card.component.ts

```
import { Component, Input, OnChanges, SimpleChanges } from '@angular/core';
```

```
@Component({
  selector: 'app-course-card',
  standalone: true,
  templateUrl: './course-card.component.html'
})
export class CourseCardComponent implements OnChanges {
  @Input() course: any;

  ngOnChanges(changes: SimpleChanges) {
    if (changes['course']) {
      console.log('Previous:', changes['course'].previousValue);
      console.log('Current:', changes['course'].currentValue);
    }
  }
}
```

course-list.component.html

```
<app-course-card [course]="{id:1,name:'Data Structures',code:'CS101',credits:4}"></app-course-card>
<app-course-card [course]="{id:2,name:'Operating Systems',code:'CS102',credits:3}"></app-course-card>
<app-course-card [course]="{id:3,name:'DBMS',code:'CS103',credits:4}"></app-course-card>
```

Task 3: @Input and @Output

course-card.component.ts

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

export class CourseCardComponent {
  @Input() course!: { id: number; name: string; code: string; credits: number };
  @Output() enrollRequested = new EventEmitter<number>();
}
```

course-card.component.html

```
<div class="card">
  <p>ID: {{ course.id }}</p>
  <p>Name: {{ course.name }}</p>
  <p>Code: {{ course.code }}</p>
  <p>Credits: {{ course.credits }}</p>
  <button (click)="enrollRequested.emit(course.id)">Enroll</button>
</div>
```

course-list.component.ts

```
export class CourseListComponent {
  selectedCourseId: number | null = null;

  courses = [
    { id: 1, name: 'Data Structures', code: 'CS101', credits: 4 },
```

```
    { id: 2, name: 'Operating Systems', code: 'CS102', credits: 3 },
    { id: 3, name: 'DBMS', code: 'CS103', credits: 4 },
    { id: 4, name: 'Computer Networks', code: 'CS104', credits: 3 },
    { id: 5, name: 'Java Programming', code: 'CS105', credits: 4 }
  ];

  onEnroll(courseId: number) {
    console.log('Enrolling in course: ' + courseId);
    this.selectedCourseId = courseId;
  }
}
```

course-list.component.html

```
<app-course-card *ngFor="let c of courses" [course]="c"
(enrollRequested)="onEnroll($event)"></app-course-card>
<p *ngIf="selectedCourseId">Selected course ID: {{ selectedCourseId }}</p>
```

HANDS-ON 3 — Directives & Pipes

Task 1: Structural Directives

course-list.component.ts

```
export class CourseListComponent implements OnInit {
  isLoading = true;

  ngOnInit() {
    setTimeout(() => { this.isLoading = false; }, 1500);
  }

  trackByCourseId(index: number, course: any) {
    return course.id;
  }
}
```

course-list.component.html

```
<p *ngIf="isLoading">Loading courses...</p>

<div *ngIf="!isLoading">
  <app-course-card
    *ngFor="let course of courses; let i = index; trackBy: trackByCourseId"
    [course]="course">
  </app-course-card>
</div>
```

```
<ng-container *ngIf="courses.length > 0; else noCourses"></ng-container>
<ng-template #noCourses><p>No courses available.</p></ng-template>
```

course-card.component.html (ngSwitch)

```
<span [ngSwitch]="course.gradeStatus">
  <span *ngSwitchCase="'passed'" class="badge green">Passed</span>
  <span *ngSwitchCase="'failed'" class="badge red">Failed</span>
  <span *ngSwitchCase="'pending'" class="badge grey">Pending</span>
</span>
```

Task 2: ngClass and ngStyle

course-card.component.ts

```
export class CourseCardComponent {
  isEnrolled = false;
  isExpanded = false;

  get cardClasses() {
    return {
      'card--enrolled': this.isEnrolled,
      'card--full': this.course.credits >= 4,
      'expanded': this.isExpanded
    };
  }

  get borderColor() {
    if (this.course.gradeStatus === 'passed') return 'green';
    if (this.course.gradeStatus === 'failed') return 'red';
    return 'grey';
  }

  toggleExpand() {
    this.isExpanded = !this.isExpanded;
  }
}
```

course-card.component.html

```
<div class="card" [ngClass]="cardClasses" [ngStyle]="{ 'border-left-color': borderColor }">
  ...
  <button (click)="toggleExpand()">Show Details</button>
</div>
```

course-card.component.css

```
.card--enrolled { background-color: #e8f5e9; }
.card--full { border-width: 2px; }
.expanded { min-height: 220px; }
```

Task 3: Custom Directive and Custom Pipe

```
ng generate directive directives/highlight
ng generate pipe pipes/credit-label
```

highlight.directive.ts

```
import { Directive, ElementRef, HostListener, Input } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true
})
export class HighlightDirective {
  @Input() appHighlight = 'yellow';

  constructor(private el: ElementRef) {}

  @HostListener('mouseenter')
  onMouseEnter() {
    this.el.nativeElement.style.backgroundColor = this.appHighlight;
  }

  @HostListener('mouseleave')
  onMouseLeave() {
    this.el.nativeElement.style.backgroundColor = '';
  }
}
```

credit-label.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'creditLabel', standalone: true })
export class CreditLabelPipe implements PipeTransform {
  transform(value: number | null): string {
    if (!value) return 'No Credits';
    return value === 1 ? '1 Credit' : value + ' Credits';
  }
}
```

usage

```
<app-course-card appHighlight="lightblue" *ngFor="let c of courses" [course]="c"></app-course-card>
<p>{{ course.credits | creditLabel }}</p>
```

Output

```
{{ 3 | creditLabel }} -> 3 Credits
{{ 1 | creditLabel }} -> 1 Credit
{{ null | creditLabel }} -> No Credits
```

HANDS-ON 4 — Template-Driven Forms & Validation

Task 1: Build the Enrollment Request Form

```
ng generate component pages/enrollment-form
```

app.routes.ts

```
{ path: 'enroll', component: EnrollmentFormComponent }
```

enrollment-form.component.ts

```
import { Component } from '@angular/core';
import { FormsModule, NgForm } from '@angular/forms';

@Component({
  selector: 'app-enrollment-form',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './enrollment-form.component.html'
})
export class EnrollmentFormComponent {
  submitted = false;

  onSubmit(form: NgForm) {
    console.log(form.value);
    console.log(form.valid);
    this.submitted = form.valid;
  }
}
```

enrollment-form.component.html

```
<form #enrollForm="ngForm" (ngSubmit)="onSubmit(enrollForm)">

  <label>Name</label>
  <input name="studentName" required minlength="3" #nameCtrl="ngModel"
[(ngModel)]="studentName">
  <span *ngIf="nameCtrl.touched && nameCtrl.errors?.['required']">Name is required</span>
  <span *ngIf="nameCtrl.touched && nameCtrl.errors?.['minlength']">Name must be at least 3
characters</span>

  <label>Email</label>
  <input name="studentEmail" type="email" required email #emailCtrl="ngModel"
[(ngModel)]="studentEmail">
  <span *ngIf="emailCtrl.touched && emailCtrl.errors?.['required']">Email is required</span>
  <span *ngIf="emailCtrl.touched && emailCtrl.errors?.['email']">Enter a valid email</span>

  <label>Course ID</label>
```

```
  <input name="courseId" type="number" required #courseCtrl="ngModel" [(ngModel)]="courseId">
  <span *ngIf="courseCtrl.touched && courseCtrl.errors?.['required']">Course ID is required</span>

  <label>Preferred Semester</label>
  <select name="preferredSemester" [(ngModel)]="preferredSemester">
    <option value="Odd">Odd</option>
    <option value="Even">Even</option>
  </select>

  <label>
    <input type="checkbox" name="agreeToTerms" required [(ngModel)]="agreeToTerms">
    I agree to the terms
  </label>

  <button type="submit" [disabled]="enrollForm.invalid">Submit</button>
  <button type="button" (click)="enrollForm.resetForm()">Reset</button>

  <div *ngIf="submitted">Enrollment request submitted successfully!</div>
</form>
```

enrollment-form.component.css

```
.ng-invalid.ng-touched { border-color: red; }
.ng-valid.ng-touched { border-color: green; }
```

HANDS-ON 5 — Reactive Forms

Task 1: Build a Reactive Form with FormBuilder

```
ng generate component pages/reactive-enrollment-form
```

app.routes.ts

```
{ path: 'enroll-reactive', component: ReactiveEnrollmentFormComponent }
```

reactive-enrollment-form.component.ts

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-reactive-enrollment-form',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './reactive-enrollment-form.component.html'
})
export class ReactiveEnrollmentFormComponent implements OnInit {
  enrollForm!: FormGroup;
```

```
  constructor(private fb: FormBuilder) {}

  ngOnInit() {
    this.enrollForm = this.fb.group({
      studentName: ['', [Validators.required, Validators.minLength(3)]],
      studentEmail: ['', [Validators.required, Validators.email]],
      courseId: [null, Validators.required],
      preferredSemester: ['Odd', Validators.required],
      agreeToTerms: [false, Validators.requiredTrue]
    });
  }

  onSubmit() {
    console.log(this.enrollForm.value);
    console.log(this.enrollForm.getRawValue());
  }
}
```

reactive-enrollment-form.component.html

```
<form [formGroup]="enrollForm" (ngSubmit)="onSubmit()">
  <input formControlName="studentName">
  <input formControlName="studentEmail">
  <input formControlName="courseId">
  <select formControlName="preferredSemester">
    <option value="Odd">Odd</option>
    <option value="Even">Even</option>
  </select>
  <input type="checkbox" formControlName="agreeToTerms">
  <button [disabled]="enrollForm.invalid">Submit</button>
</form>
```

Task 2: Custom Validators and FormArray

validators/course-code.validator.ts

```
import { AbstractControl, ValidationErrors } from '@angular/forms';

export function noCourseCode(control: AbstractControl): ValidationErrors | null {
  return control.value && control.value.toString().startsWith('XX')
    ? { noCourseCode: true }
    : null;
}
```

reactive-enrollment-form.component.ts

```
import { noCourseCode } from '../../validators/course-code.validator';
import { AbstractControl } from '@angular/forms';

courseId: [null, [Validators.required, noCourseCode]],
```

```
simulateEmailCheck(control: AbstractControl) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(control.value && control.value.includes('test@') ? { emailTaken: true } : null);
    }, 800);
  });
}

get additionalCourses() {
  return this.enrollForm.get('additionalCourses') as FormArray;
}

addCourse() {
  this.additionalCourses.push(this.fb.control('', Validators.required));
}

removeCourse(i: number) {
  this.additionalCourses.removeAt(i);
}
```

ngOnInit addition

```
studentEmail: this.fb.control('', [Validators.required, Validators.email], [this.simulateEmailCheck]),
additionalCourses: this.fb.array([])
```

template

```
<span *ngIf="enrollForm.get('courseId')?.errors?.['noCourseCode']">Course code starting with XX is not allowed.</span>

<div *ngFor="let ctrl of additionalCourses.controls; let i = index">
  <input [formControl]="ctrl">
  <button (click)="removeCourse(i)">Remove</button>
</div>
<button type="button" (click)="addCourse()">Add Another Course</button>
```

Getter is used instead of casting in the template because a template expression cannot use the as keyword, and the getter keeps type-casting logic in the TypeScript file.

HANDS-ON 6 — Services & Dependency Injection

Task 1: Course Service

```
ng generate service services/course
```

models/course.model.ts

```
export interface Course {
  id: number;
  name: string;
  code: string;
  credits: number;
  gradeStatus: 'passed' | 'failed' | 'pending';
}
```

course.service.ts

```
import { Injectable } from '@angular/core';
import { Course } from '../models/course.model';

@Injectable({ providedIn: 'root' })
export class CourseService {
  private courses: Course[] = [
    { id: 1, name: 'Data Structures', code: 'CS101', credits: 4, gradeStatus: 'passed' },
    { id: 2, name: 'Operating Systems', code: 'CS102', credits: 3, gradeStatus: 'pending' },
    { id: 3, name: 'DBMS', code: 'CS103', credits: 4, gradeStatus: 'passed' },
    { id: 4, name: 'Computer Networks', code: 'CS104', credits: 3, gradeStatus: 'failed' },
    { id: 5, name: 'Java Programming', code: 'CS105', credits: 4, gradeStatus: 'pending' }
  ];

  getCourses(): Course[] {
    return this.courses;
  }

  getCourseById(id: number): Course | undefined {
    return this.courses.find(c => c.id === id);
  }

  addCourse(course: Course): void {
    this.courses.push(course);
  }
}
```

course-list.component.ts

```
constructor(private courseService: CourseService) {}

ngOnInit() {
  this.courses = this.courseService.getCourses();
}
```

home.component.ts

```
constructor(private courseService: CourseService) {}

get coursesAvailable() {
  return this.courseService.getCourses().length;
}
```

Task 2: Enrollment Service and Hierarchical DI

```
ng generate service services/enrollment
```

enrollment.service.ts

```
import { Injectable } from '@angular/core';
import { CourseService } from './course.service';
import { Course } from '../models/course.model';

@Injectable({ providedIn: 'root' })
export class EnrollmentService {
  private enrolledCourseIds: number[] = [];

  constructor(private courseService: CourseService) {}

  enroll(courseId: number): void {
    if (!this.enrolledCourseIds.includes(courseId)) {
      this.enrolledCourseIds.push(courseId);
    }
  }

  unenroll(courseId: number): void {
    this.enrolledCourseIds = this.enrolledCourseIds.filter(id => id !== courseId);
  }

  isEnrolled(courseId: number): boolean {
    return this.enrolledCourseIds.includes(courseId);
  }

  getEnrolledCourses(): Course[] {
    return this.enrolledCourseIds
      .map(id => this.courseService.getCourseById(id))
      .filter(c => c !== undefined) as Course[];
  }
}
```

course-card.component.ts

```
constructor(private enrollmentService: EnrollmentService) {}

toggleEnroll() {
  this.enrollmentService.isEnrolled(this.course.id)
    ? this.enrollmentService.unenroll(this.course.id)
    : this.enrollmentService.enroll(this.course.id);
}

get buttonLabel() {
  return this.enrollmentService.isEnrolled(this.course.id) ? 'Unenroll' : 'Enroll';
}
```

student-profile.component.ts

```
constructor(private enrollmentService: EnrollmentService) {}

enrolledCourses = this.enrollmentService.getEnrolledCourses();
```

notification.component.ts

```
@Component({
  selector: 'app-notification',
  standalone: true,
  providers: [NotificationService],
  templateUrl: './notification.component.html'
})
export class NotificationComponent {}
```

HANDS-ON 7 — Angular Routing

Task 1: Route Configuration, Parameters and Nested Routes

```
ng generate component pages/course-detail
ng generate component pages/not-found
```

app.routes.ts

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'courses',
    component: CoursesLayoutComponent,
    children: [
      { path: '', component: CourseListComponent },
      { path: ':id', component: CourseDetailComponent }
    ]
  },
  { path: 'profile', component: StudentProfileComponent, canActivate: [authGuard] },
  { path: '**', component: NotFoundComponent }
];
```

course-detail.component.ts

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { CourseService } from '../../services/course.service';

export class CourseDetailComponent implements OnInit {
  course: any;

  constructor(private route: ActivatedRoute, private courseService: CourseService) {}
```

```
  ngOnInit() {
    const id = Number(this.route.snapshot.paramMap.get('id'));
    this.course = this.courseService.getCourseById(id);
  }
}
```

course-list.component.ts

```
constructor(private router: Router, private route: ActivatedRoute) {}

goToDetail(course: any) {
  this.router.navigate(['courses', course.id]);
}

ngOnInit() {
  this.searchTerm = this.route.snapshot.queryParamMap.get('search') || '';
}

onSearch() {
  this.router.navigate(['courses'], { queryParams: { search: this.searchTerm } });
}
```

courses-layout.component.html

```
<router-outlet></router-outlet>
```

Task 2: Lazy Loading and Route Guards

```
ng generate module features/enrollment --routing
ng generate guard guards/auth
ng generate guard guards/unsaved-changes
```

app.routes.ts

```
{
  path: 'enroll',
  loadChildren: () => import('./features/enrollment/enrollment.module').then(m =>
m.EnrollmentModule),
  canActivate: [authGuard]
}
```

guards/auth.guard.ts

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn) {
```

```
  return true;
 }
 router.navigate(['/']);
 return false;
};
```

guards/unsaved-changes.guard.ts

```
import { CanDeactivateFn } from '@angular/router';
import { ReactiveEnrollmentFormComponent } from '../pages/reactive-enrollment-form/reactive-enrollment-form.component';

export const unsavedChangesGuard: CanDeactivateFn<ReactiveEnrollmentFormComponent> = (component) => {
 if (component.enrollForm.dirty) {
  return window.confirm('You have unsaved changes. Leave?');
 }
 return true;
};
```

Network tab observation

```
enrollment-module.js chunk downloaded only when /enroll is visited the first time, not on initial load.
```

HANDS-ON 8 — HTTP Client

Setup

```
npm install -g json-server
json-server --watch db.json --port 3000
```

db.json

```
{
 "courses": [
  { "id": 1, "name": "Data Structures", "code": "CS101", "credits": 4, "gradeStatus": "passed" },
  { "id": 2, "name": "Operating Systems", "code": "CS102", "credits": 3, "gradeStatus": "pending" }
 ],
 "students": [],
 "enrollments": []
}
```

Task 1: Replace Service Data with HttpClient Calls

course.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Course } from '../models/course.model';
```

```
@Injectable({ providedIn: 'root' })
export class CourseService {
  private apiUrl = 'http://localhost:3000/courses';

  constructor(private http: HttpClient) {}

  getCourses(): Observable<Course[]> {
    return this.http.get<Course[]>(this.apiUrl);
  }

  getCourseById(id: number): Observable<Course> {
    return this.http.get<Course>(`${this.apiUrl}/${id}`);
  }

  createCourse(course: Omit<Course, 'id'>): Observable<Course> {
    return this.http.post<Course>(this.apiUrl, course);
  }

  updateCourse(id: number, course: Partial<Course>): Observable<Course> {
    return this.http.put<Course>(`${this.apiUrl}/${id}`, course);
  }

  deleteCourse(id: number): Observable<any> {
    return this.http.delete(`${this.apiUrl}/${id}`);
  }
}
```

course-list.component.ts

```
ngOnInit() {
  this.courseService.getCourses().subscribe({
    next: courses => this.courses = courses,
    error: err => this.errorMessage = err.message,
    complete: () => this.isLoading = false
  });
}
```

app.config.ts

```
import { provideHttpClient } from '@angular/common/http';

providers: [provideHttpClient()]
```

Task 2: RxJS Operators and Error Handling

course.service.ts

```
import { map, catchError, tap, retry, switchMap } from 'rxjs/operators';
import { throwError } from 'rxjs';
```

```
getCourses(): Observable<Course[]> {
  return this.http.get<Course[]>(this.apiUrl).pipe(
    map(courses => courses.filter(c => c.credits > 0)),
    tap(courses => console.log('Courses loaded:', courses.length)),
    retry(2),
    catchError(err => {
      console.error(err);
      return throwError(() => new Error('Failed to load courses. Please try again.'));
    })
  );
}
// tap is used only for logging side effects and never mutates the stream, unlike map which
// transforms the emitted value.
// switchMap cancels the previous inner Observable subscription when a new courseId arrives,
// preventing an outdated response from overwriting a newer one.
```

enrollment.service.ts

```
getStudentsByCourse(courseId: number): Observable<any[]> {
  return this.http.get<any[]>(`http://localhost:3000/enrollments?courseId=${courseId}`);
}
```

course-detail.component.ts

```
this.route.paramMap.pipe(
  switchMap(params => this.enrollmentService.getStudentsByCourse(Number(params.get('id'))))
).subscribe(students => this.students = students);
```

Task 3: HTTP Interceptors

```
ng generate interceptor interceptors/auth
ng generate interceptor interceptors/error-handler
ng generate interceptor interceptors/loading
```

interceptors/auth.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const cloned = req.clone({ setHeaders: { Authorization: 'Bearer mock-token-12345' } });
  return next(cloned);
};
```

interceptors/error-handler.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';

export const errorHandlerInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  return next(req).pipe(
```

```
    catchError(err => {
      if (err.status === 401) {
        router.navigate(['/']);
      }
      if (err.status === 500) {
        console.error('Server error, please try again later');
      }
      return throwError(() => err);
    })
  );
};
```

services/loading.service.ts

```
import { Injectable } from '@angular/core';
import { BehaviorSubject } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class LoadingService {
  isLoading$ = new BehaviorSubject<boolean>(false);
  show() { this.isLoading$.next(true); }
  hide() { this.isLoading$.next(false); }
}
```

interceptors/loading.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { finalize } from 'rxjs';
import { LoadingService } from '../services/loading.service';

export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loadingService = inject(LoadingService);
  loadingService.show();
  return next(req).pipe(finalize(() => loadingService.hide()));
};
```

app.config.ts

```
provideHttpClient(withInterceptors([authInterceptor, errorHandlerInterceptor, loadingInterceptor]))
```

app.component.html

```
<div class="spinner" *ngIf="loadingService.isLoading$ | async">Loading...</div>
```

HANDS-ON 9 — NgRx Store, Actions, Reducers, Effects & Selectors

Install

```
npm install @ngrx/store @ngrx/effects @ngrx/entity @ngrx/store-devtools
```

Task 1: Set Up NgRx Store

app.config.ts

```
import { provideStore } from '@ngrx/store';
import { provideStoreDevtools } from '@ngrx/store-devtools';
import { courseReducer } from './store/course/course.reducer';

providers: [
  provideStore({ course: courseReducer }),
  provideStoreDevtools({ maxAge: 25 })
]
```

store/course/course.actions.ts

```
import { createAction, props } from '@ngrx/store';
import { Course } from '../../models/course.model';

export const loadCourses = createAction('[Course] Load Courses');
export const loadCoursesSuccess = createAction('[Course] Load Courses Success', props<{ courses: Course[] }>());
export const loadCoursesFailure = createAction('[Course] Load Courses Failure', props<{ error: string }>());
```

store/course/course.reducer.ts

```
import { createReducer, on } from '@ngrx/store';
import { Course } from '../../models/course.model';
import { loadCourses, loadCoursesSuccess, loadCoursesFailure } from './course.actions';

export interface CourseState {
  courses: Course[];
  loading: boolean;
  error: string | null;
}

const initialState: CourseState = { courses: [], loading: false, error: null };

export const courseReducer = createReducer(
  initialState,
  on(loadCourses, state => ({ ...state, loading: true, error: null })),
  on(loadCoursesSuccess, (state, { courses }) => ({ ...state, courses, loading: false })),
  on(loadCoursesFailure, (state, { error }) => ({ ...state, error, loading: false }))
);
```

store/course/course.selectors.ts

```
import { createFeatureSelector, createSelector } from '@ngrx/store';
import { CourseState } from './course.reducer';

export const selectCourseState = createFeatureSelector<CourseState>('course');
export const selectAllCourses = createSelector(selectCourseState, state => state.courses);
export const selectCoursesLoading = createSelector(selectCourseState, state => state.loading);
export const selectCoursesError = createSelector(selectCourseState, state => state.error);
```

course-list.component.ts

```
constructor(private store: Store) {}

courses$ = this.store.select(selectAllCourses);

ngOnInit() {
  this.store.dispatch(loadCourses());
}
```

course-list.component.html

```
<app-course-card *ngFor="let c of courses$ | async" [course]="c"></app-course-card>
```

Task 2: NgRx Effects

store/course/course.effects.ts

```
import { Injectable } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { switchMap, map, catchError } from 'rxjs/operators';
import { of } from 'rxjs';
import { CourseService } from '../../services/course.service';
import { loadCourses, loadCoursesSuccess, loadCoursesFailure } from './course.actions';

@Injectable()
export class CourseEffects {
  loadCourses$ = createEffect(() =>
    this.actions$.pipe(
      ofType(loadCourses),
      switchMap(() =>
        this.courseService.getCourses().pipe(
          map(courses => loadCoursesSuccess({ courses })),
          catchError(error => of(loadCoursesFailure({ error: error.message })))
        )
      )
    )
  );

  constructor(private actions$: Actions, private courseService: CourseService) {}
}
```

app.config.ts

```
import { provideEffects } from '@ngrx/effects';
providers: [provideEffects([CourseEffects])]
```

store/enrollment/enrollment.actions.ts

```
import { createAction, props } from '@ngrx/store';

export const enrollInCourse = createAction('[Enrollment] Enroll', props<{ courseId: number }>());
export const unenrollFromCourse = createAction('[Enrollment] Unenroll', props<{ courseId: number
}>());
export const setEnrolledCourses = createAction('[Enrollment] Set Enrolled', props<{ ids: number[]
}>());
```

store/enrollment/enrollment.reducer.ts

```
import { createReducer, on } from '@ngrx/store';
import { enrollInCourse, unenrollFromCourse, setEnrolledCourses } from './enrollment.actions';

export interface EnrollmentState {
  enrolledCourseIds: number[];
}

const initialState: EnrollmentState = { enrolledCourseIds: [] };

export const enrollmentReducer = createReducer(
  initialState,
  on(enrollInCourse, (state, { courseId }) => ({
    enrolledCourseIds: [...state.enrolledCourseIds, courseId]
  })),
  on(unenrollFromCourse, (state, { courseId }) => ({
    enrolledCourseIds: state.enrolledCourseIds.filter(id => id !== courseId)
  })),
  on(setEnrolledCourses, (state, { ids }) => ({ enrolledCourseIds: ids }))
);
```

store/enrollment/enrollment.selectors.ts

```
import { createFeatureSelector, createSelector } from '@ngrx/store';
import { EnrollmentState } from './enrollment.reducer';
import { selectAllCourses } from '../course/course.selectors';

export const selectEnrollmentState = createFeatureSelector<EnrollmentState>('enrollment');
export const selectEnrolledIds = createSelector(selectEnrollmentState, state => state.enrolledCourseIds);
export const selectEnrolledCourses = createSelector(
  selectAllCourses,
  selectEnrolledIds,
  (courses, ids) => courses.filter(c => ids.includes(c.id))
);
```

course-card.component.ts

```
constructor(private store: Store) {}

enrolledIds$ = this.store.select(selectEnrolledIds);

onEnrollClick() {
  this.store.dispatch(enrollInCourse({ courseId: this.course.id }));
}
```

course-card.component.html

```
<button (click)="onEnrollClick()">
  {{ (enrolledIds$ | async)?.includes(course.id) ? 'Unenroll' : 'Enroll' }}
</button>
```

HANDS-ON 10 — Unit Testing

Task 1: Testing CourseCardComponent

course-card.component.spec.ts

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { By } from '@angular/platform-browser';
import { CourseCardComponent } from './course-card.component';

describe('CourseCardComponent', () => {
  let component: CourseCardComponent;
  let fixture: ComponentFixture<CourseCardComponent>;

  const mockCourse = { id: 1, name: 'Data Structures', code: 'CS101', credits: 4, gradeStatus: 'passed' as
const };

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [CourseCardComponent]
    });
    fixture = TestBed.createComponent(CourseCardComponent);
    component = fixture.componentInstance;
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should display course name from input', () => {
    component.course = mockCourse;
    fixture.detectChanges();
    const el = fixture.debugElement.query(By.css('h3')).nativeElement;
    expect(el.textContent).toContain('Data Structures');
```

```
  });

  it('should emit enrollRequested on button click', () => {
    component.course = mockCourse;
    fixture.detectChanges();
    spyOn(component.enrollRequested, 'emit');
    fixture.debugElement.query(By.css('button')).nativeElement.click();
    fixture.detectChanges();
    expect(component.enrollRequested.emit).toHaveBeenCalledWith(1);
  });

  it('should log on ngOnChanges', () => {
    spyOn(console, 'log');
    component.ngOnChanges({
      course: {
        previousValue: null,
        currentValue: mockCourse,
        firstChange: true,
        isFirstChange: () => true
      }
    } as any);
    expect(console.log).toHaveBeenCalled();
  });

  it('should apply card--full class when credits >= 4', () => {
    component.course = mockCourse;
    fixture.detectChanges();
    expect(component.cardClasses['card--full']).toBeTrue();
  });
});
```

Test run output

```
CourseCardComponent
  should create
  should display course name from input
  should emit enrollRequested on button click
  should log on ngOnChanges
  should apply card--full class when credits >= 4

5 specs, 0 failures
```

Task 2: Testing CourseService and an NgRx-Connected Component

course.service.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
import { CourseService } from './course.service';

describe('CourseService', () => {
```

```
  let service: CourseService;
  let httpMock: HttpTestingController;

  const mockCourses = [
    { id: 1, name: 'Data Structures', code: 'CS101', credits: 4, gradeStatus: 'passed' },
    { id: 2, name: 'Operating Systems', code: 'CS102', credits: 3, gradeStatus: 'pending' }
  ];

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
      providers: [CourseService]
    });
    service = TestBed.inject(CourseService);
    httpMock = TestBed.inject(HttpTestingController);
  });

  afterEach(() => {
    httpMock.verify();
  });

  it('should fetch courses', () => {
    service.getCourses().subscribe(courses => {
      expect(courses.length).toBe(2);
    });
    const req = httpMock.expectOne('http://localhost:3000/courses');
    expect(req.request.method).toBe('GET');
    req.flush(mockCourses);
  });

  it('should handle 500 error', () => {
    service.getCourses().subscribe({
      error: err => expect(err.message).toContain('Failed to load courses')
    });
    const req = httpMock.expectOne('http://localhost:3000/courses');
    req.flush('Server error', { status: 500, statusText: 'Internal Server Error' });
  });
});
```

course-list.component.spec.ts (MockStore)

```
import { TestBed } from '@angular/core/testing';
import { provideMockStore, MockStore } from '@ngrx/store/testing';
import { By } from '@angular/platform-browser';
import { CourseListComponent } from './course-list.component';
import { selectAllCourses } from '../../store/course/course.selectors';

describe('CourseListComponent (NgRx)', () => {
  let fixture: any;
  let store: MockStore;
```

```
  const mockCourses = [
    { id: 1, name: 'Data Structures', code: 'CS101', credits: 4, gradeStatus: 'passed' }
  ];

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [CourseListComponent],
      providers: [
        provideMockStore({
          initialState: { course: { courses: mockCourses, loading: false, error: null } }
        })
      ]
    });
    store = TestBed.inject(MockStore);
    fixture = TestBed.createComponent(CourseListComponent);
  });

  it('should render course cards from initial state', () => {
    fixture.detectChanges();
    const cards = fixture.debugElement.queryAll(By.css('app-course-card'));
    expect(cards.length).toBe(1);
  });

  it('should show loading indicator when loading is true', () => {
    store.setState({ course: { courses: [], loading: true, error: null } });
    fixture.detectChanges();
    const loadingEl = fixture.debugElement.query(By.css('.loading'));
    expect(loadingEl).toBeTruthy();
  });
});
```

Test run output

```
CourseService
  should fetch courses
  should handle 500 error

CourseListComponent (NgRx)
  should render course cards from initial state
  should show loading indicator when loading is true

4 specs, 0 failures
```